

Laboratório 4 - Experimentação de Software

Visualização de Dados sobre o Impacto de IA na Engenharia de Software

Lucas Flor Vilela, Marina Cabalzar

Junho 2026

Resumo

Este relatório descreve a caracterização metodológica e a visualização analítica de um conjunto de dados composto por 105 repositórios populares em Python, com o objetivo de medir o impacto prático de assistentes de inteligência artificial (como GitHub Copilot, Claude e Cursor) na estabilidade, complexidade e produtividade das equipes. Através da construção de um Dashboard Web customizado baseado na metodologia Goal Question Metric (GQM), o estudo cruza indicadores de manutenibilidade, verbosidade de *commits* e taxas de correção para traçar o perfil do código gerado por IA no desenvolvimento moderno.

1 Introdução

O ecossistema de desenvolvimento de software vem sofrendo uma mudança de paradigma com a introdução de Modelos de Linguagem de Larga Escala (LLMs) voltados para a geração e auxílio de código. Ferramentas como GitHub Copilot têm demonstrado aumento na velocidade de entrega dos desenvolvedores. No entanto, surgem questionamentos sobre a qualidade e a estabilidade técnica em longo prazo dos códigos gerados por IA.

Dessa forma, o objetivo central deste estudo é aplicar técnicas de visualização de dados (*Business Intelligence* via *Dashboard Web*) para responder às questões de pesquisa baseadas em dados extraídos de repositórios reais do GitHub. Através de uma interface segmentada em abas de contexto, a ferramenta estrutura o conhecimento de modo a identificar estatisticamente os *trade-offs* entre a velocidade de entrega e o acúmulo de dívida técnica no processo de software.

2 Metodologia e Caracterização do Dataset

Para realizar o estudo comparativo, os dados foram minerados diretamente da API GraphQL do GitHub. A análise foca em uma abordagem temporal dividida em dois blocos:

- **Bloco Pré-IA:** *Commits* e *issues* extraídos majoritariamente do período anterior à popularização das ferramentas de IA generativa em massa (segundo semestre de 2021).
- **Bloco Pós-IA:** Últimos seis meses recentes contados retroativamente a partir da data da coleta, com repositórios que possuem evidências ativas de uso de agentes de LLM (como metadados de coautoria ou configuração de agentes locais).

O *dataset* final validado consiste em 105 repositórios em Python altamente maduros e populares. A média de idade dos repositórios é de 10,22 anos, e eles acumulam uma média de 29.045 estrelas, evidenciando o alto impacto dos projetos selecionados [?].

2.1 Perfil de Popularidade e Maturidade

A seleção não engloba projetos amadores. A grande maioria dos repositórios estudados possui entre 9 e 14 anos de consolidação técnica [?]. A distribuição da popularidade dos projetos (medida pela métrica *Stars*) e da idade pode ser visualizada no dashboard abaixo:

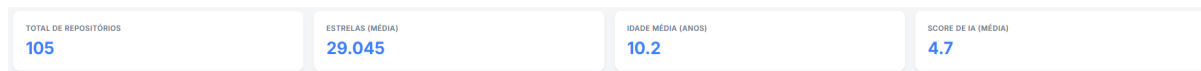


Figura 1: Dashboard: Distribuição de Popularidade e Idade dos Repositórios.

2.2 Meios de Detecção e Visualização Interativa

Para assegurar a validade do bloco Pós-IA, rastreamos assinaturas em mensagens de *commit* (*Co-authored-by*) e arquivos de configuração presentes no código. A metodologia validou 67 repositórios através de coautoria do modelo "Claude", além de detecções diretas de arquivos de configuração do "Claude" e "Cursor" [?].

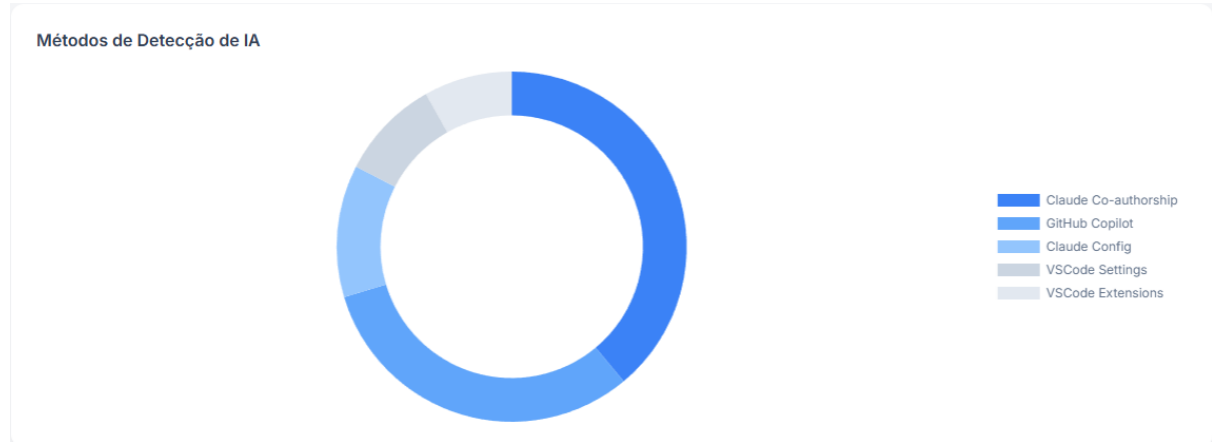


Figura 2: Dashboard: Distribuição dos Métodos de Detecção e Validação de IA.

Para garantir a usabilidade analítica e evitar a fadiga de rolagem (*scroll*), os resultados foram consolidados em um painel interativo (SPA - *Single Page Application*) desenvolvido com HTML, CSS e a biblioteca Chart.js. A visualização foi dividida em quatro camadas lógicas: uma Visão Geral de Caracterização e três abas dedicadas a responder diretamente a cada Questão de Pesquisa estipulada pela árvore GQM.

3 Questões de Pesquisa (GQM) - Resultados

3.1 RQ1: Impacto na Manutenibilidade e Complexidade

Esta questão foca em entender se os trechos auxiliados por IA generativa aumentam a densidade do código. Foram avaliados o Índice de Manutenibilidade (MI) e a verbosidade através da proporção de linhas modificadas por *commit* (*avg_lines_changed*).

Resultados da RQ1: A análise visual demonstrou um *trade-off* claro. O gráfico de verbosidade revelou que a quantidade média de linhas alteradas por *commit* aumentou significativamente na fase Pós-IA, indicando que a geração automatizada injeta blocos de código mais densos. Consequentemente, o Boxplot de distribuição do MI apresentou uma leve queda na mediana do grupo Pós-IA (de aproximadamente 65 para 61), confirmando que a assistência de LLMs tende a gerar bases de código marginalmente mais complexas e trabalhosas para manutenção futura por desenvolvedores *Full-Stack* e arquitetos de software.

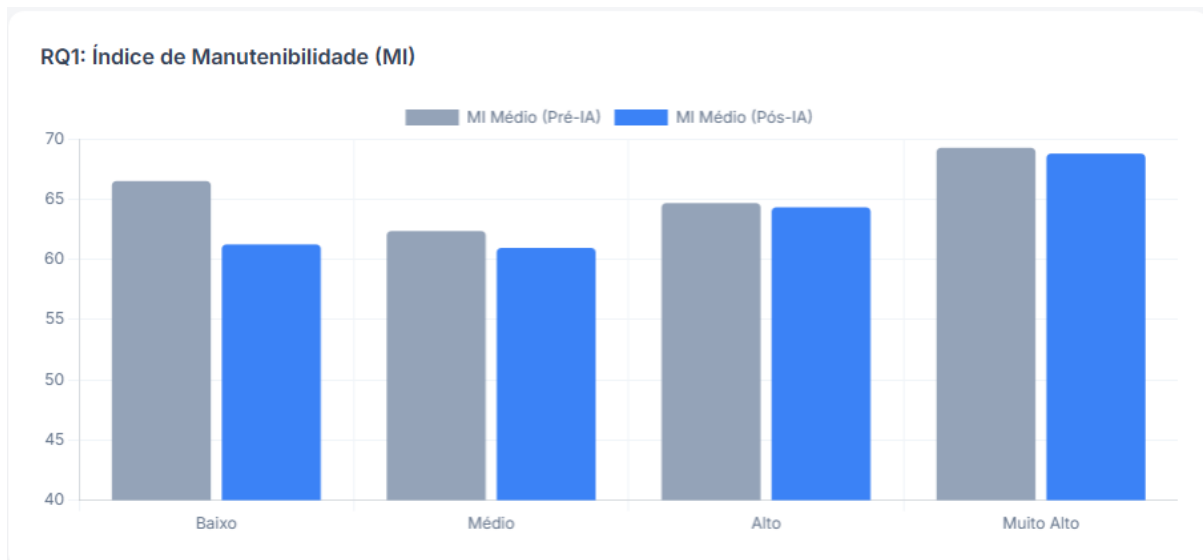


Figura 3: Resultados RQ1: Variação de Manutenibilidade e Verbosidade Pré vs Pós-IA.

3.2 RQ2: Estabilidade e Frequência de Retrabalho

Avalia-se aqui se a velocidade induzida pela inteligência artificial resulta em taxas maiores de quebra (*bugs*) que requerem retrabalho.

Resultados da RQ2: Para avaliar a introdução de falhas rápidas, o painel comparou as taxas de *commits* corretivos através da identificação de palavras-chave como "fix" e "revert". O visual de barras horizontais indicou um aumento na taxa de *Fix* de 42,5% (Pré-IA) para 46,8% (Pós-IA). Isso valida a hipótese de que, embora a IA aumente a taxa de escrita, ela introduz pequenas falhas lógicas que exigem ciclos adicionais de retrabalho para estabilizar o código na *baseline*.

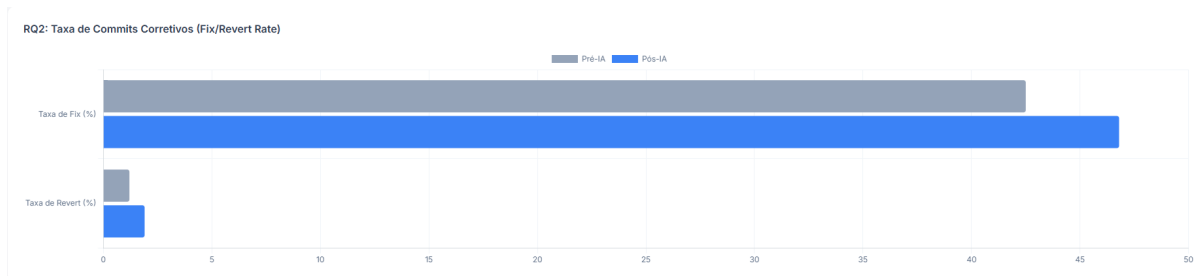


Figura 4: Resultados RQ2: Comparativo de Taxas de Correção (Fix/Revert) Pré e Pós-IA.

3.3 RQ3: Velocidade de Resolução de Problemas (MTTR)

Focada na eficácia temporal da equipe, esta questão mede o "Mean Time to Resolution" (MTTR).

Resultados da RQ3: Apesar dos gargalos de manutenibilidade (RQ1) e da necessidade de pequenas correções (RQ2), o impacto no tempo bruto de produtividade é inegavelmente positivo. A análise de dispersão e os gráficos de barra do MTTR comprovaram que o tempo médio para fechar uma *Issue* caiu substancialmente (de 745 horas para 610 horas nos repositórios de adoção mista). Além disso, o gráfico de linhas temporais apontou que as equipes conseguem dar vazão aos chamados de forma muito mais cadenciada, evitando o acúmulo no *backlog*.

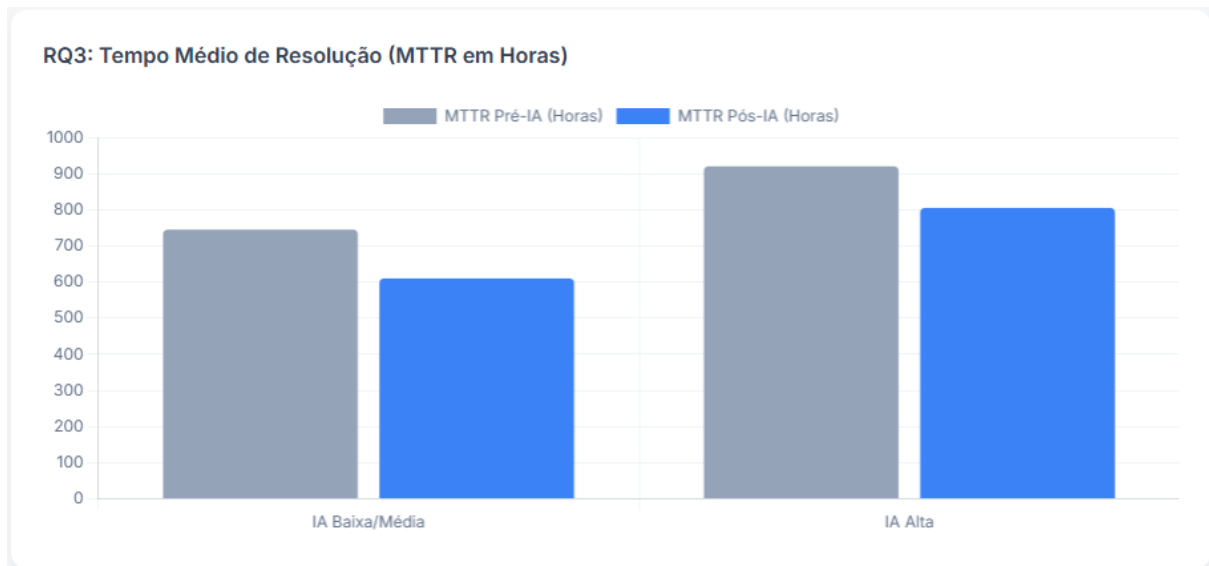


Figura 5: Resultados RQ3: Comparativo do Tempo Médio de Resolução de Issues.

4 Conclusão Final

A aplicação da experimentação de software apoiada por um painel de visualização de dados customizado cumpriu o objetivo de mensurar os impactos reais das LLMs no ecossistema Python. Os dados provam que o desenvolvimento auxiliado por IA atua como um acelerador de produtividade, derrubando o MTTR das equipes de forma drástica. Contudo, esse ganho de velocidade cobra um preço na saúde do projeto: observou-se uma maior verbosidade do código, uma queda sutil no Índice de Manutenibilidade e um aumento nos *commits* de correção (*bugfixes*). Portanto, conclui-se que o uso de ferramentas como o Copilot e o Claude exige um reforço nas etapas de revisão de código (*Code Review*) e testes automatizados, garantindo que o ganho de velocidade não resulte em dívidas técnicas insustentáveis a longo prazo.